BeaconKit 아키텍처 분석 - 09. 데이터 가용성 레이어

1. 데이터 가용성 개요

BeaconKit의 데이터 가용성(DA) 레이어는 **EIP-4844 (Proto-Danksharding)**를 구현하여 블롭(blob) 데이터의 가용성을 보장합니다. 이는 이더리움의 확장성 솔루션의 핵심 구성 요소로, 롤업과 같은 L2 솔루션이 대량의 데이터를 효율적으로 저장할 수 있게 합니다.

1.1 핵심 개념

블롭 (Blob):

- 최대 128KB의 데이터 청크
- 트랜잭션 실행에 직접 접근 불가
- KZG 커밋먼트로 암호학적으로 커밋됨

KZG 커밋먼트:

- 블롭 데이터에 대한 다항식 커밋먼트
- 데이터 무결성 보장
- 효율적인 증명 생성 가능

블롭 사이드카:

- 블롭 데이터와 관련 증명들의 패키지
- 네트워크를 통해 별도로 전파
- 일정 기간 후 자동 정리

1.2 DA 레이어 구조

```
┌─────────────────────────────────────────────────────────────┐
| Data Availability Layer |
├─────────────────┬─────────────────┬──────────────────────────┤
| BlobFactory | BlobProcessor | BlobVerifier |
| (사이드카 생성) | (사이드카 처리) | (KZG 증명 검증) |
├─────────────────┼─────────────────┼──────────────────────────┤
| KZG Prover | AvailabilityStore | Merkle Proofs |
| (증명 생성) | (블롭 저장) | (포함 증명) |
└─────────────────┴─────────────────┴──────────────────────────┘
```

2. BlobFactory - 사이드카 생성

2.1 SidecarFactory 구조

```go
type SidecarFactory struct {
// metrics는 팩토리 메트릭 수집 및 보고에 사용
metrics *factoryMetrics
}

func  NewSidecarFactory(telemetrySink TelemetrySink) *SidecarFactory {
return &SidecarFactory{
metrics: newFactoryMetrics(telemetrySink),
}
```

```
}
```

2.2 사이드카 빌딩 프로세스

```
func (f *SidecarFactory) BuildSidecars(
signedBlk *ctypes.SignedBeaconBlock,
bundle engineprimitives.BlobsBundle,
) (types.BlobSidecars, error) {
var (
blobs = bundle.GetBlobs() // 실제 블롭 데이터
commitments = bundle.GetCommitments() // KZG 커밋먼트들
proofs = bundle.GetProofs() // KZG 증명들
numBlobs = uint64(len(blobs))
sidecars = make([]*types.BlobSidecar, numBlobs)
blk = signedBlk.GetBeaconBlock()
body = blk.GetBody()
header = blk.GetHeader()
g = errgroup.Group{}
)

startTime := time.Now()
defer  f.metrics.measureBuildSidecarsDuration(startTime, math.U64(numBlobs))
```

```go
// SignedBeaconBlock의 서명 재사용
// 서명 검증 시 해당 BeaconBlock으로 서명 루트 재구성 필요
sigHeader := ctypes.NewSignedBeaconBlockHeader(header, signedBlk.GetSignature())


// 각 블롭에 대해 병렬로 사이드카 생성
for i := range numBlobs {
g.Go(func() error {
// KZG 포함 증명 생성
inclusionProof, err := f.BuildKZGInclusionProof(body, math.U64(i))
if err != nil {
return err
}
// 블롭 사이드카 구성
sidecars[i] = types.BuildBlobSidecar(
math.U64(i), // 인덱스
sigHeader, // 서명된 헤더
blobs[i], // 블롭 데이터
commitments[i], // KZG 커밋먼트
proofs[i], // KZG 증명
inclusionProof, // 포함 증명
)
return nil
})
```

```
}

return sidecars, g.Wait()
}
```

2.3 KZG 포함 증명 생성

```
func (f *SidecarFactory) BuildKZGInclusionProof(
body *ctypes.BeaconBlockBody,
index math.U64,
) ([]common.Root, error) {
startTime := time.Now()
defer f.metrics.measureBuildKZGInclusionProofDuration(startTime)

// STEP 1: 커밋먼트 리스트 내에서 커밋먼트에 대한 머클 증명 구축
commitmentsProof, err := f.BuildCommitmentProof(body, index)
if err != nil {
return nil, err
}

// STEP 2: 블록 바디 루트에 대한 머클 증명 구축
bodyProof, err := f.BuildBlockBodyProof(body)
if err != nil {
```

```go
return nil, err
}

// STEP 3: 머클 트리의 속성에 의해 두 증명을 연결하여 최종 증명 생성
return append(commitmentsProof, bodyProof...), nil
}
```

2.4 블록 바디 증명 생성

```go
func (f *SidecarFactory) BuildBlockBodyProof(
body *ctypes.BeaconBlockBody,
) ([]common.Root, error) {
startTime := time.Now()
defer f.metrics.measureBuildBlockBodyProofDuration(startTime)
// 최상위 레벨 루트들 획득
tlrs, err := body.GetTopLevelRoots()
if err != nil {
return nil, err
}
// 최대 리프 수로 머클 트리 생성
tree, err := merkle.NewTreeWithMaxLeaves[common.Root](
tlrs,
body.Length()-1,
```

```go
)
if err != nil {
return nil, err
}

// KZG 위치에 대한 머클 증명 생성
return tree.MerkleProof(ctypes.KZGPosition)
}
```

2.5 커밋먼트 증명 생성

```go
func (f *SidecarFactory) BuildCommitmentProof(
body *ctypes.BeaconBlockBody,
index math.U64,
) ([]common.Root, error) {
startTime := time.Now()
defer f.metrics.measureBuildCommitmentProofDuration(startTime)
// KZG 커밋먼트들로 머클 트리 생성
bodyTree, err := merkle.NewTreeWithMaxLeaves[common.Root](
body.GetBlobKzgCommitments().Leafify(),
constants.MaxBlobCommitmentsPerBlock,
)
if err != nil {
```

```
return  nil, err
}

// 특정 인덱스에 대한 머클 증명 생성 (믹신 포함)
return  bodyTree.MerkleProofWithMixin(index.Unwrap())
}
```

3. BlobProcessor - 사이드카 처리

3.1 Processor 구조

```
type Processor struct {
// chainSpec은 체인 사양
chainSpec ChainSpec
// verifier는 블롭 증명 검증자
verifier BlobProofVerifier
// metrics는 프로세서 메트릭
metrics *processorMetrics
}
```

3.2 사이드카 처리 프로세스

```
func (sp *Processor) ProcessSidecars(
```

```go
avs AvailabilityStore,
sidecars types.BlobSidecars,
) error {
startTime := time.Now()
defer sp.metrics.measureProcessSidecarsDuration(startTime, math.U64(len(sidecar


// 각 사이드카 처리
for _, sidecar := range sidecars {
if err := sp.processSidecar(avs, sidecar); err != nil {
return err
}
}
return nil
}


func (sp *Processor) processSidecar(
avs AvailabilityStore,
sidecar *types.BlobSidecar,
) error {
// STEP 1: 사이드카를 가용성 저장소에 저장
if err := avs.Persist(sidecar.GetSignedBeaconBlockHeader().GetSlot(), sidecar);
return err
}
```

```
// STEP 2: 추가 검증 로직 (필요시)
return nil
}
```

3.3 사이드카 검증

```
func (sp *Processor) VerifySidecars(
ctx context.Context,
sidecars types.BlobSidecars,
blkHeader *ctypes.BeaconBlockHeader,
kzgCommitments eip4844.KZGCommitments[common.ExecutionHash],
) error {
startTime := time.Now()
defer sp.metrics.measureVerifySidecarsDuration(startTime, math.U64(len(sidecars

// STEP 1: 사이드카 수와 커밋먼트 수 일치 확인
if len(sidecars) != len(kzgCommitments) {
return ErrSidecarCommitmentMismatch
}

// STEP 2: 각 사이드카 개별 검증
for i, sidecar := range sidecars {
```

```
if  err := sp.verifySidecar(ctx, sidecar, blkHeader, kzgCommitments[i]);  err !=
return  err
}
}


// STEP 3: 배치 KZG 증명 검증 (효율성을 위해)
return  sp.verifyBlobProofBatch(sidecars)
}
```

4. KZG 증명 시스템

4.1 BlobProofVerifier 인터페이스

```
type BlobProofVerifier interface {
// GetImplementation은 검증자의 구현을 반환
GetImplementation() string
// VerifyBlobProof는 블롭 데이터가 제공된 커밋먼트에 해당하는지 검증
VerifyBlobProof(
blob *eip4844.Blob,
proof eip4844.KZGProof,
commitment eip4844.KZGCommitment,
) error
// VerifyBlobProofBatch는 여러 증명을 배치로 검증 (더 효율적)
```

```
VerifyBlobProofBatch(*kzgtypes.BlobProofArgs) error
}
```

4.2 KZG 검증자 생성

```
func NewBlobProofVerifier(
impl string,
ts *gokzg4844.JSONTrustedSetup,
) (BlobProofVerifier, error) {
switch impl {
case gokzg.Implementation:
return gokzg.NewVerifier(ts)
default:
return nil, errors.Wrapf(
ErrUnsupportedKzgImplementation,
"supplied: %s, supported: %s",
impl, gokzg.Implementation,
)
}
}
```

4.3 배치 증명 검증

```go
func ArgsFromSidecars(scs datypes.BlobSidecars) *kzgtypes.BlobProofArgs {
proofArgs := &kzgtypes.BlobProofArgs{
Blobs: make([]*eip4844.Blob, len(scs)),
Proofs: make([]eip4844.KZGProof, len(scs)),
Commitments: make([]eip4844.KZGCommitment, len(scs)),
}
for i, sidecar := range scs {
blob := sidecar.GetBlob()
proofArgs.Blobs[i] = &blob
proofArgs.Proofs[i] = sidecar.GetKzgProof()
proofArgs.Commitments[i] = sidecar.GetKzgCommitment()
}
return proofArgs
}
```

5. 블롭 사이드카 구조

5.1 BlobSidecar 타입

```go
type BlobSidecar struct {
// index는 블록 내에서 블롭의 인덱스
index math.U64
// signedBeaconBlockHeader는 서명된 비콘 블록 헤더
```

```
signedBeaconBlockHeader *ctypes.SignedBeaconBlockHeader

// blob은 실제 블롭 데이터

blob eip4844.Blob

// kzgCommitment는 블롭에 대한 KZG 커밋먼트

kzgCommitment eip4844.KZGCommitment

// kzgProof는 블롭에 대한 KZG 증명

kzgProof eip4844.KZGProof

// kzgCommitmentInclusionProof는 커밋먼트가 블록에 포함되었음을 증명

kzgCommitmentInclusionProof []common.Root

}
```

5.2 사이드카 빌딩

```
func  BuildBlobSidecar(

index math.U64,

signedBeaconBlockHeader *ctypes.SignedBeaconBlockHeader,

blob eip4844.Blob,

kzgCommitment eip4844.KZGCommitment,

kzgProof eip4844.KZGProof,

kzgCommitmentInclusionProof []common.Root,

) *BlobSidecar {

return &BlobSidecar{

index: index,
```

```
signedBeaconBlockHeader: signedBeaconBlockHeader,

blob: blob,

kzgCommitment: kzgCommitment,

kzgProof: kzgProof,

kzgCommitmentInclusionProof: kzgCommitmentInclusionProof,

}

}
```

6. AvailabilityStore - 블롭 저장소

6.1 저장소 인터페이스

```
type AvailabilityStore interface {

// Persist는 블롭 사이드카를 저장

Persist(slot math.Slot, sidecar *types.BlobSidecar) error

// Get은 특정 슬롯과 인덱스의 블롭 사이드카를 검색

Get(slot math.Slot, index  uint64) (*types.BlobSidecar, error)

// IsDataAvailable은 블록의 모든 블롭 데이터가 사용 가능한지 확인

IsDataAvailable(ctx context.Context, slot math.Slot, body BeaconBlockBody) bool

// Prune은 DA 기간을 벗어난 블롭들을 정리

Prune(currentSlot math.Slot) error

}
```

6.2 데이터 가용성 확인

```go
func (as *Store) IsDataAvailable(
ctx context.Context,
slot math.Slot,
body BeaconBlockBody,
) bool {
commitments := body.GetBlobKzgCommitments()
// 모든 KZG 커밋먼트에 대해 해당 블롭 사이드카 존재 확인
for  i, commitment := range  commitments {
if !as.HasBlobSidecar(slot, uint64(i), commitment) {
return false
}
}
return true
}
```

6.3 TTL 기반 정리

```go
func (as *Store) Prune(currentSlot math.Slot) error {
// DA 기간 계산 (일반적으로 4096 에포크)
cutoffSlot := currentSlot - as.chainSpec.MinEpochsForBlobSidecarsRequest() * as.
// 컷오프 슬롯 이전의 모든 블롭 삭제
```

```
return as.deleteBeforeSlot(cutoffSlot)
}
```

7. 머클 증명 시스템

7.1 포함 증명 구조

블롭 사이드카의 포함 증명은 두 부분으로 구성됩니다:

1. 커밋먼트 증명:

- KZG 커밋먼트가 커밋먼트 리스트에 포함되었음을 증명
- 머클 트리의 특정 인덱스에 대한 증명

2. 블록 바디 증명:

- 커밋먼트 리스트가 블록 바디에 포함되었음을 증명
- 블록 바디의 최상위 레벨 구조에 대한 증명

7.2 증명 검증

```
func VerifyInclusionProof(
commitment eip4844.KZGCommitment,
proof []common.Root,
blockRoot common.Root,
index uint64,
) bool {
// 1. 커밋먼트에서 시작하여 증명 경로를 따라 올라감
```

```go
// 2. 각 단계에서 형제 노드와 해시

// 3. 최종적으로 블록 루트와 일치하는지 확인

currentHash := commitment.Hash()

for _, sibling := range proof {

if index%2 == 0 {

currentHash = hash(currentHash, sibling)

} else {

currentHash = hash(sibling, currentHash)

}

index /= 2

}

return currentHash == blockRoot

}
```

8. 성능 최적화

8.1 병렬 처리

사이드카 생성:

```go
// 각 블롭에 대해 병렬로 사이드카 생성

for i := range numBlobs {

g.Go(func() error {

// 포함 증명 생성 및 사이드카 구성
```

```
return nil
})
}
return sidecars, g.Wait()
```

배치 검증:

```
// 여러 KZG 증명을 한 번에 검증 (단일 검증보다 효율적)
func (sp *Processor) verifyBlobProofBatch(sidecars types.BlobSidecars) error {
args := ArgsFromSidecars(sidecars)
return sp.verifier.VerifyBlobProofBatch(args)
}
```

8.2 캐싱 전략

머클 트리 캐싱:

- 자주 사용되는 머클 트리 구조 캐시
- 증명 생성 시간 단축

KZG 설정 캐싱:

- Trusted Setup 데이터 메모리 캐시
- 검증 성능 향상

9. 메트릭 및 모니터링

9.1 팩토리 메트릭

```go
type factoryMetrics struct {
buildSidecarsDuration prometheus.Histogram
buildKZGInclusionProofDuration prometheus.Histogram
buildBlockBodyProofDuration prometheus.Histogram
buildCommitmentProofDuration prometheus.Histogram
}
```

9.2 프로세서 메트릭

```go
type processorMetrics struct {
processSidecarsDuration prometheus.Histogram
verifySidecarsDuration prometheus.Histogram
blobProofVerifications prometheus.Counter
blobProofFailures prometheus.Counter
}
```

9.3 검증자 메트릭

```go
type verifierMetrics struct {
singleProofVerificationTime prometheus.Histogram
batchProofVerificationTime prometheus.Histogram
proofVerificationSuccesses prometheus.Counter
proofVerificationFailures prometheus.Counter
```

```
}
```

10. 오류 처리

10.1 KZG 관련 오류

```
var (
ErrInvalidKZGCommitment = errors.New("invalid KZG commitment")
ErrInvalidKZGProof = errors.New("invalid KZG proof")
ErrKZGVerificationFailed = errors.New("KZG verification failed")
ErrUnsupportedKzgImplementation = errors.New("unsupported KZG implementation")
)
```

10.2 사이드카 관련 오류

```
var (
ErrSidecarCommitmentMismatch = errors.New("sidecar commitment mismatch")
ErrSidecarIndexOutOfRange = errors.New("sidecar index out of range")
ErrSidecarNotFound = errors.New("sidecar not found")
ErrInclusionProofInvalid = errors.New("inclusion proof invalid")
)
```

11. 설계 원칙

11.1 암호학적 안전성

- KZG 커밋먼트를 통한 데이터 무결성
- 머클 증명을 통한 포함 증명
- Trusted Setup 기반 보안

11.2 효율성

- 배치 검증을 통한 성능 향상
- 병렬 처리를 통한 처리량 증대
- 캐싱을 통한 중복 계산 방지

11.3 확장성

- TTL 기반 자동 정리
- 모듈형 검증자 구조
- 설정 가능한 DA 기간

11.4 호환성

- EIP-4844 표준 준수
- 이더리움 생태계와의 호환성
- 다양한 KZG 구현 지원

이 데이터 가용성 레이어는 BeaconKit이 대규모 데이터를 효율적으로 처리하면서도 암호학적 보안을 유지할 수 있게 해주는 핵심 시스템입니다.